

OVERVIEW · MAY 2026

The runtime where production AI agents actually work.

A versioned workspace, a tool gateway, multi-agent channels. Designed for AI agents. Not retrofitted from human UIs.

What Plinth is, in plain English.

Imagine you hired a freelance researcher to write you a report. Two ways to manage them.

WAY A

The phone call

You keep them on a phone call the whole time. Every time they want to remember something, they read it back to you aloud. Every new source, they read aloud. By hour three the call is enormous, every minute costs a fortune, and they've half-forgotten what they decided in hour one.

→ This is how most AI agents work today.

WAY B

The desk

You give them a desk. They put notes down, pick them up, file research, label folders, bookmark pages. When they need something, they look it up. The desk has drawers. The drawers have labels. Three hours in, the desk has structure, and they remember what they decided in hour one because it is written on the desk.

→ This is Plinth.

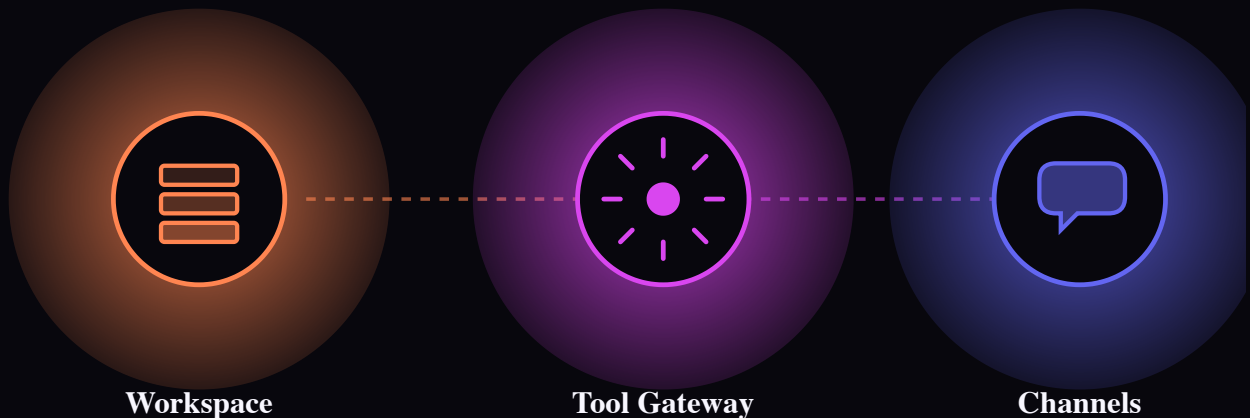
Plinth is the desk for AI agents. It is the part of the system that holds onto what the agent has learned, fetched, and decided — so the agent doesn't have to keep saying it out loud, over and over, to itself.

The phone-call problem is not theoretical. A naive 10-step research workflow sends the entire prior conversation back to the model with every step. By step 10 you're paying ten times what step one cost, every time. The bill scales with the square of the work — which is why most "production agents" are actually three-step demos that fall over the moment you ask them to do something serious.

Plinth replaces the phone-call architecture with the desk architecture. Same agent, same model, same output. About 71% fewer tokens. Measured, not marketed.

Three primitives. One coherent surface.

Each one removes a category of plumbing every agent team rebuilds. They are designed together, not retrofitted.



01 · PERSISTENT WORKSPACE

Key-value store, files, snapshots, branches. Every write is versioned. The agent's memory survives crashes, restarts, and 50-step workflows. Take a labelled checkpoint before a risky tool call; restore in one line if it goes wrong.

02 · TOOL GATEWAY

Single boundary for every external tool. OAuth, caching, audit log, cost caps. The agent calls `tools.invoke("web.search", ...)` and never sees a credential. Eight providers built-in: GitHub, Slack, Linear, Notion, Google, Atlassian, Salesforce, Asana.

03 · MULTI-AGENT CHANNELS

Durable typed message queues between specialist agents. Researcher writes drafts; Writer reads them and composes; Reviewer reads and approves. Each agent keeps a narrow prompt. No agent has to know what the others are doing.

The same agent, but the bill stops growing quadratically.

71%

FEWER TOKENS

Measured on a 5-source research task.
Reproducible from a clean clone via make demo.

\$60→\$3

30-PDF TASK

Same model. Same output quality. The workspace stores docs; the prompt only carries handles and summaries.

~Linear

COST VS WORK

Chat-history architecture is quadratic. Workspace architecture is roughly linear. The gap widens with every step.

The mechanism in one sentence

Instead of re-sending the entire prior conversation with every model call, the agent gets back **handles** — short strings like `ws://my-research/sources/renewables.txt@v17` — and dereferences them only when needed, in slices.

CONCRETE EXAMPLE · 30 PDFS RESEARCH TASK

STEP	CHAT-HISTORY WAY	PLINTH WAY
1 Fetch all PDFs	600k tokens in chat history	30 handles in workspace, prompt ~3 KB
2 Summarise each	Prompt: 600k + "now summarise"	30 × 5 KB excerpts → 300-token summaries
3 Synthesise themes	Prompt: 605k + "now compare"	Prompt: ~12 KB (just the summaries)
∑ Total tokens	~6,000,000 token-equivalents	~200,000 token-equivalents
\$ Total cost	\$60 — \$180	\$1 — \$3

One SDK call. The runtime does the rest.

The same primitives in Python, TypeScript, Go, Swift, and Kotlin. API v1 is locked: additive-only for at least 12 months.

```
# Python · the canonical SDK
from plinth import Plinth

client = Plinth(
    workspace_url="http://localhost:7421",
    gateway_url="http://localhost:7422",
    api_key="local-dev",
)

ws = client.workspace("my-research")
ws.kv.set("topic", "renewable energy")

# Stored in workspace, doesn't bloat prompt history
sources = client.tools.invoke("web.search", {"query": "renewable energy"})
for src in sources["results"]:
    content = client.tools.invoke("web.fetch", {"url": src["url"]}) # cached
    ws.files.write(f"sources/{slug(src['url'])}.txt", content)

ws.snapshot("sources-collected") # named checkpoint
```

What the SDK gives you

- **Handles, not values.** Reads return URIs; the model only sees content on explicit dereference.
- **Auto-projections.** Summaries, vector indexes, and outlines generated on write — async, no blocking.
- **Branch & merge.** Fork the workspace, try three approaches in parallel, keep the winner.
- **Lease-based workers.** Crash mid-flight, pick up where you left off. Verified by a runnable demo.

The contract you get

- **API v1 is locked.** Additive-only for 12+ months. Breaking changes go to a /v2/ namespace with migration guide.
- **Apache 2.0.** The Cloud product runs identical code. No "community edition" with the good parts ripped out.
- **2,867 tests green.** Three SDKs in Python/TS/Go to API parity; Swift and Kotlin to feature parity in v2.
- **Threat model published.** 90-day fix window on vulnerabilities. Bounty matrix on /security.

A single coherent surface, composed from independent services.

Each service runs as its own process. Replace, scale, or harden any one of them independently. No hidden coupling. No magic.



AGENT SIDE

SDK in five languages. One call, no glue code.

PLINTH RUNTIME

Workspace + Gateway + Identity. All replaceable.

TOOLS SIDE

Eight OAuth providers built-in. MCP for the rest.

PRICING

Free for solo. Pro at \$99. Enterprise on request.

The runtime is Apache 2.0 — fully self-hostable. Cloud is the managed version, same code we run, you don't operate it.

POPULAR		
Free	Pro	Enterprise
\$0 / month	\$99 / month	Custom
For solo builders and OSS projects.	For production-agent teams.	For Fortune-500 / regulated teams.
5 workspaces	Unlimited workspaces	Dedicated tenancy or on-prem
10k tool calls / mo	500k tool calls / mo	BYOK encryption
2 GB storage	100 GB storage	SOC 2 + DPA + MSA
Community support	Audit log retention 12 months	Custom integrations
	Email support, 24h response	Solutions engineer + SLA

How to get started in 3 minutes

SELF-HOSTED

```
# macOS · Linux
$ curl -fsSL
https://plinth.dev/install.sh | sh

# Dashboard opens at
http://localhost:7420
```

CLOUD (WHEN READY)

Sign up at plinth.dev/signup, create your first workspace, paste your LLM key, run the sample task. Five minutes to a working demo with token-cost numbers you can take to your CTO.

Three doors.

FOR EVERYONE

Read the manifesto.

Three pages on why agents deserve their own runtime, written for product people who don't want to learn what a vector store is.

plinth.dev/manifesto

FOR ENGINEERS

Clone the repo and run the demo.

One command produces the 71% reduction number on your own laptop. The benchmark code is in the repo. The threat model is in the repo. The architecture decisions are in the repo as ADRs.

github.com/plinth/plinth

FOR TEAMS

Talk to us.

If you're running production agents and the bill is climbing, we'd like to hear what's on fire. We run a 30-minute call, you share traces, we tell you honestly whether Plinth helps. No deck.

hello@plinth.dev



Plinth

The runtime where production AI agents actually work.

● v1.7 · API v1 stable